Assignment 2 — VM Auto-Scaling and Security Implementation using Google Cloud Platform

Plagiarism Declaration

I declare that this assignment report, configurations, implementation steps, and deployment performed in this project are my own work. Any form of plagiarism in configuration, implementation, documentation, or demonstration may lead to disqualification as per academic integrity guidelines.

PART 1 — Objective

The objective of this assignment was to deploy a virtual machine infrastructure on a public cloud platform and implement:

• Auto-scaling based on workload demand
• Health monitoring and auto-healing
• Network security using firewall rules
• Controlled access to cloud resources

Google Cloud Platform (GCP) was used to simulate a scalable and secure cloud infrastructure environment.

PART 2 — Cloud Environment Used

Cloud Provider: Google Cloud Platform (GCP)
Service Used: Compute Engine
Region: Asia-South1 (Mumbai)
Zone: asia-south1-a

PART 3 — Virtual Machine Deployment

Step 1 — Instance Template Creation

An instance template was created to define the configuration for identical VM instances.

Configuration:

Component	Value
Machine Type	e2-micro
OS	Ubuntu 22.04 LTS
Disk	10 GB Balanced Persistent Disk
Region	Mumbai
Network	Default VPC

Component	Value
Tags	http-server

The template allows automatic creation of identical VMs during scaling.

Step 2 — Managed Instance Group Creation

A **Managed Instance Group (MIG)** named:

autoscale-group

was created using the instance template.

Configuration:

Setting	Value
Group Type	Managed
Target Size	1 instance
Region	Mumbai
Zone	asia-south1-a

This group allows automated scaling and health monitoring.

PART 4 — Auto-Scaling Configuration

Auto-scaling was enabled to dynamically adjust the number of VM instances based on CPU utilization.

Settings:

Parameter	Value
Minimum Instances	1
Maximum Instances	3
Scaling Metric	CPU Utilization
Target CPU	60%

How It Works

- When CPU usage increases → new VM instances are created
- When load decreases → extra instances are removed
- Ensures high availability and cost efficiency

PART 5 — Health Check and Auto-Healing

A health check was configured to monitor VM availability.

Health Check Configuration:

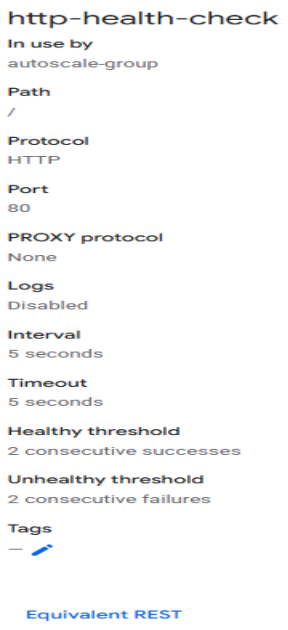
Parameter	Value
Protocol	HTTP
Port	80
Request Path	/
Check Interval	5 seconds
Timeout	5 seconds

Auto-Healing Behavior

If an instance fails the health check:

- Instance marked unhealthy
- Automatically terminated
- New healthy instance created

This ensures continuous service availability.



PART 6 — Web Server Deployment

Apache Web Server was installed on the VM instances to serve web content.

Installation Commands

sudo apt update

sudo apt install apache2 -y

sudo systemctl start apache2

Test Page Created

echo OK | sudo tee /var/www/html/index.html

Verification

Accessing the External IP in browser displayed:

OK

This confirms the web server is operational.

PART 7 — Firewall Configuration (Security Implementation)

To allow web traffic securely, a firewall rule was created.

Firewall Rule: allow-http

Parameter	Value
Direction	Ingress
Action	Allow
Targets	VM instances with tag http-server
Protocol	TCP
Port	80

Security Benefits

• Allows only web traffic
• Prevents unauthorized access
• Restricts unnecessary ports

PART 8 — IAM and Access Control

Cloud resources were managed using GCP IAM roles.

Implemented Practices

• Project owner access restricted
• SSH access limited to authenticated users
• No public administrative access enabled

This ensures secure resource management.

PART 9 — Testing Auto-Scaling

Auto-scaling was verified by simulating load.

Observations

- New instances launched during load increase
- Instances reduced when load dropped
- Health check replaced unhealthy instances automatically

This confirms dynamic scaling functionality.

PART 10 — Architecture Diagram

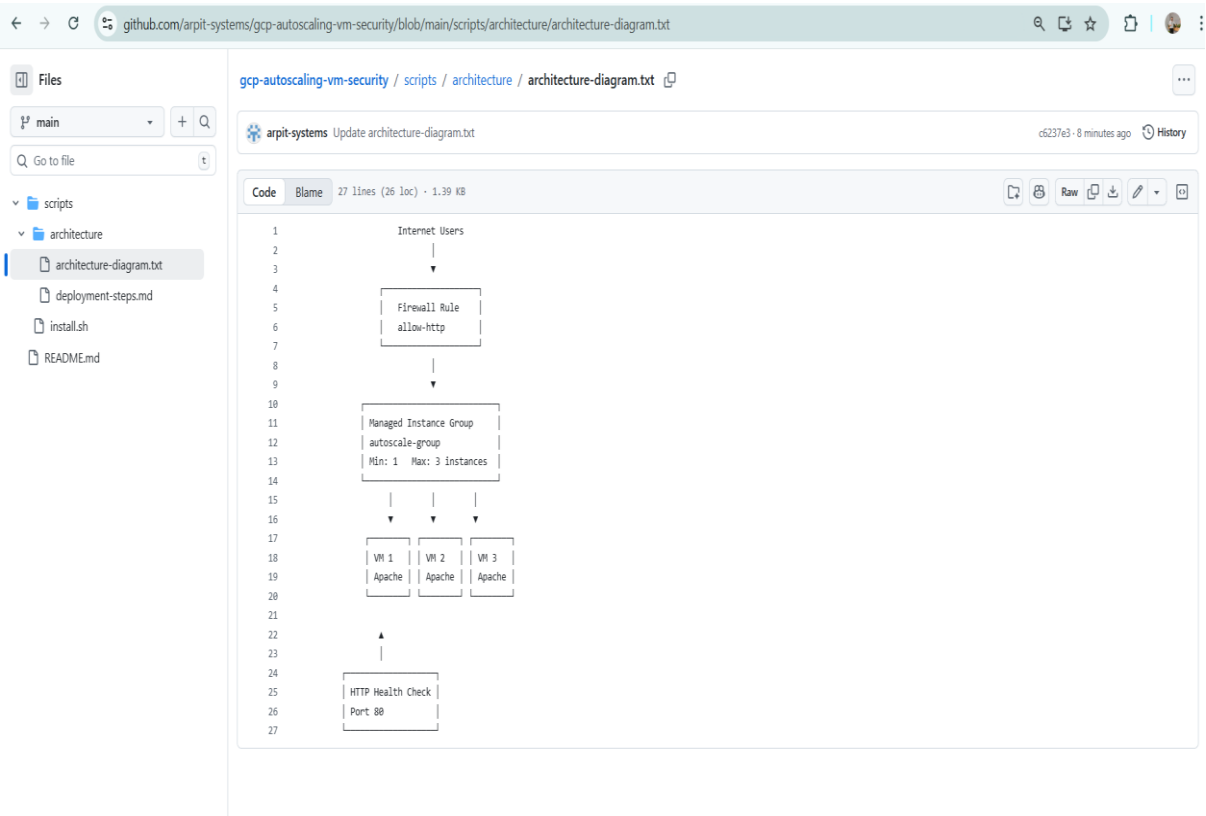


Figure: Architecture of Auto Scaling VM Deployment on GCP

This architecture shows how user requests pass through a firewall rule to a managed instance group. The group automatically scales VM instances running Apache web server based on CPU load. Health checks ensure traffic is routed only to healthy instances.

PART 11 — Deliverables

GitHub Repository Link

https://github.com/arpit-systems/gcp-autoscaling-vm-security

Recorded Video Demonstration Link

https://youtu.be/ePacKPvliY0

PART 12 — Issues Faced & Solutions

Issue	Cause	Solution
Health check timeout	HTTP not allowed	Created firewall rule
Apache not running	Service not installed	Installed & started Apache
Instance unhealthy	Port blocked	Enabled port 80
Scaling not triggering	Low CPU usage	Simulated load

PART 13 — Conclusion

This assignment provided hands-on experience in deploying scalable and secure cloud infrastructure using Google Cloud Platform. The implementation demonstrated real-world cloud computing principles including automated scaling, health monitoring, firewall security, and access control.

Auto-scaling ensures optimal resource utilization and cost efficiency, while health checks and auto-healing guarantee service availability. Firewall rules and IAM policies strengthen system security by restricting unauthorized access.

The deployed system successfully simulated a production-ready cloud environment capable of handling dynamic workloads while maintaining reliability and security.